

## Final Project Writeup

### Overview

My project utilizes 2024-2025 NBA player data to aggregate performance statistics and determine players' primary roles based on total points, assists, rebounds, and defensive "stocks" (steals + blocks). The user can view top performers in a given category or look up players' stats by team.

The dataset I used is from Kaggle, called "NBA Player Game Logs (2024-2025)," created by Eduardo Palmieri. The dataset contains over 16,000 rows of player statistics. If a player was traded throughout the season, their stats are aggregated across teams.

### Data Processing

I loaded the dataset with the csv crate and deserialized it into a custom `PlayerGame` struct using `serde`. Missing values (e.g., players who did not record a stat in a game) are handled using `Option<f32>` types and substituted with a 0.0 when aggregating. All actual analysis is done in Rust. No preprocessing beyond Rust was used.

### Code Structure

My project is modular and separated into four Rust files:

`main.rs`: Handles all user input and control flow.

- `main()`: Main program entry point. Displays a numbered menu to the user, processes input, and calls one of two helper functions depending on the user's choice.
- `team_player_flow()`: Asks the user to select a team, then a player within that team, and finally prints the complete stat breakdown and rankings of the player using formatted output and rank lookups.
- `leaderboard_flow()`: Prompts the user to select a stat category and then displays the top 20 players for the selected stat from pre-sorted leaderboard vectors.

`data.rs`: Handles data loading and raw struct definition.

- `PlayerGame`: Struct that represents the performance of a single player in a single game. Every stat (e.g., points, assists) is an `Option<f32>` to accommodate missing values.
- `load_players()`: Loads the CSV files using the csv crate, deserializes each row into a `PlayerGame`, and returns a vector of all player entries using an iterator and error handling.

`stats.rs`: Handles stat aggregation and role assignment.

- `PlayerStats`: Struct that stores every player's season totals for points, assists, rebounds, steals, blocks, games played, and their team's abbreviation.
- `aggregate_stats()`: Iterates through a slice of `PlayerGame` entries and builds a `HashMap` keyed by player name, accumulating totals by summing each stat and counting games played.
- `assign_primary_role()`: Determines a player's "primary role" by comparing their ranks on four stat leaderboards and selecting the category where they are ranked the highest.

**leaderboard.rs**: Computes sorted rankings and determines player leaderboard positions.

- **build\_leaderboards()**: Iterates over all combined player stats and creates sorted vectors for points, assists, rebounds, and defensive stocks by cloning and sorting the relevant values.
- **find\_rank()**: Finds a player's name in a statistics leaderboard and returns his rank as a formatted string or reports if he is not ranked.

Modules are imported into **main.rs** using **mod** and **use** declarations. Modules all have documentation comments and have been made for readability and reuse.

## Tests

**data.rs** tests:

- **test\_load\_players\_valid\_csv()**  
Purpose: Ensure a valid CSV file is correctly parsed into the **PlayerGame** struct.  
Significance: It confirms that my primary data loading function works correctly and can be trusted to load my program with clean data.
- **test\_load\_players\_invalid\_path()**  
Purpose: Ensure that the program correctly returns an error when trying to load a non-existent file.  
Significance: It tests my error handling and prevents runtime crashes on invalid file paths.

**stats.rs** tests:

- **test\_aggregate\_stats\_single\_game()**  
Purpose: Checks that a single **PlayerGame** is correctly aggregated into **PlayerStats**.  
Significance: Verifies my stat aggregation logic correctly returns totals and behaves with optional values as expected.
- **test\_assign\_primary\_role\_returns\_expected()**  
Purpose: Checks that the assignment of the primary role assignment function chooses the correct role given rank position on all leaderboards.  
Significance: Validates that my program logically assigns a player's role.

**leaderboard.rs** tests:

- **test\_build\_leaderboards\_sorts\_correctly()**  
Purpose: Verifies that all stat leaderboards are correctly sorted in descending order and by the correct metric.  
Significance: Ensures the rankings shown to the user are accurate and logically sorted.
- **test\_find\_rank\_returns\_correct\_index()**  
Purpose: Checks that the function returns the correct rank of a player if they are in the leaderboard, or "Not Ranked" otherwise.  
Significance: Ensures leaderboard indexing is working and users receive correct rankings.

```
PS C:\Users\sara\OneDrive\CDSDS_210\project\project> cargo test
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.05s
Running unittests src/main.rs (target/debug/deps/project-b4acd0bf0b9301b9.exe)

running 6 tests
test data::tests::test_load_players_invalid_path ... ok
test leaderboard::tests::test_build_leaderboards_sorts_correctly ... ok
test leaderboard::tests::test_find_rank_returns_correct_index ... ok
test stats::tests::test_assign_primary_role_returns_expected ... ok
test stats::tests::test_aggregate_stats_single_game ... ok
test data::tests::test_load_players_valid_csv ... ok

test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

## Example Output & Interpretation

```
PS C:\Users\sara\OneDrive\CDS\210\project\project> cargo run
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.04s
Running `target\debug\project.exe`

(NBA player stats are current through February 7, 2025.)
(If a player was traded during the season, their stats are combined across teams.)

Choose an option:
1. Select a player by team
2. View NBA leaderboards for a stat
1

Available teams:
ATL BOS BRK CHI CHO CLE DAL DEN DET GSW HOU IND LAC LAL MEM MIA MIL MIN NOP NYK OKC ORL PHI PHO POR SAC SAS TOR UTA WAS

Enter a team abbreviation (e.g., BOS, LAL, NYK):
BOS

Players for BOS:
1. Al Horford
2. Baylor Scheierman
3. Derrick White
4. Drew Peterson
5. JD Davison
6. Jaden Springer
7. Jaylen Brown
8. Jayson Tatum
9. Jordan Walsh
10. Jrue Holiday
11. Kristaps Porzingis
12. Luke Kornet
13. Neemias Queta
14. Payton Pritchard
15. Sam Hauser
16. Xavier Tillman
17. Xavier Tillman Sr.

Enter the number next to the player you want to view:
8

Selected: Jayson Tatum
- Total Points: 1301 (Rank 6 in NBA)
- Total Assists: 277 (Rank 19 in NBA)
- Total Rebounds: 424 (Rank 16 in NBA)
- Total Steals + Blocks: 86 (Rank 48 in NBA)
Primary Role: Scorer
```

Interpretation: This example shows that the team and player selection flow works smoothly, and the user is given an informative summary that includes rankings and the player's primary role.

```
PS C:\Users\sara\OneDrive\CDS\210\project\project> cargo run
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.04s
Running `target\debug\project.exe`

(NBA player stats are current through February 7, 2025.)
(If a player was traded during the season, their stats are combined across teams.)

Choose an option:
1. Select a player by team
2. View NBA leaderboards for a stat
2

Choose a stat to view leaderboard:
points, assists, rebounds, defense
points

Top players for points:
1. Shai Gilgeous-Alexander - 1622
2. Anthony Edwards - 1388
3. Nikola Jokić - 1366
4. Giannis Antetokounmpo - 1328
5. Jalen Brunson - 1304
6. Jayson Tatum - 1301
7. Tyrese Maxey - 1225
8. Devin Booker - 1215
9. Cade Cunningham - 1203
10. Donovan Mitchell - 1177
11. De'Aaron Fox - 1170
12. Tyler Herro - 1163
13. LeBron James - 1125
14. Karl-Anthony Towns - 1119
15. Trae Young - 1116
16. Jalen Green - 1094
17. Darius Garland - 1093
18. Anthony Davis - 1081
19. Damian Lillard - 1075
20. Victor Wembanyama - 1050
```

Interpretation: This confirms that the leaderboard for total points is correctly sorted in descending order and includes real player names with realistic totals.

```

PS C:\Users\sara\OneDrive\CDS\210\project\project> cargo run
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s
Running `target\debug\project.exe`

(NBA player stats are current through February 7, 2025.)
(If a player was traded during the season, their stats are combined across teams.)

Choose an option:
1. Select a player by team
2. View NBA leaderboards for a stat
1

Available teams:
ATL BOS BRK CHI CHO CLE DAL DEN DET GSW HOU IND LAC LAL MEM MIA MIL MIN NOP NYK OKC ORL PHI PHO POR SAC SAS TOR UTA WAS

Enter a team abbreviation (e.g., BOS, LAL, NYK):
SEA
No players found for team SEA

```

Interpretation: This demonstrates the program’s ability to handle invalid input gracefully. In this case, the user tried entering “SEA” for the former Seattle SuperSonics, and the program correctly responded with a message indicating that the team does not exist in the dataset.

## Usage Instructions

To build and run my project, navigate the project directory in the terminal and enter:

```

cargo build
cargo run

```

Once the program runs, the user is prompted with two numbered options:

```

Choose an option:
1. Select a player by team
2. View NBA leaderboards for a stat

```

- If 1 is selected, the program will ask the user to enter a team abbreviation (e.g., BOS or OKC). Input is case-insensitive, so either uppercase or lowercase entries will work. The user will then choose a player based on the corresponding number.
- If 2 is selected, the user will be prompted to type one of the following keywords: **points**, **assists**, **rebounds**, or **defense**. The program will then output the top 20 players in the selected category.

The program includes input validation and prints helpful error messages for invalid selections.

Runtime: The program compiles and executes efficiently, processing over 16,000 rows of data in well under one second on standard hardware. No special optimizations or long runtimes are expected.

## AI Assistance & Citations

- In `team_player_flow()`, I collected all team abbreviations into a `Vec<String>` and removed duplicates using `teams.sort()` followed by `teams.dedup()`.
- This ensured that each team appeared only once in the list shown to the user.
- I understood that `.dedup()` only removes adjacent duplicates, which is why sorting beforehand was necessary.

I have a list of team abbreviations and some are duplicated. What's the simplest way to remove duplicates from a `Vec<String>` in Rust?

To remove duplicates from a `Vec<String>` in Rust, the simplest way is to first **sort the vector**, then call `.dedup()`:

```
rust
my_vec.sort();
my_vec.dedup();
```

This works because `.dedup()` only removes **consecutive** duplicates, so sorting ensures duplicates are grouped together.

If keeping the original order matters, you'd need to use a `HashSet` to track seen values and filter them out manually — but for many cases, sort + dedup is the easiest method.

- I created four modules: `data.rs` for CSV loading, `stats.rs` for stat aggregation, `leaderboard.rs` for ranking logic, and `main.rs` for program control and user interaction.
- Each module contains only relevant functions and structs (e.g., `PlayerGame` lives in `data.rs`, `PlayerStats` in `stats.rs`), following single-responsibility principles.
- In `main.rs`, I use `mod` and `use` statements to access these modules cleanly, reducing clutter and improving code organization.

I'm working on a Rust project that processes and analyzes NBA player stats. I have functions for reading data, aggregating stats, ranking players, and displaying results. How should I split this into separate modules?

Great question! In Rust, it's common to split code into modules based on **logical responsibilities**. For your case, here's a clean way to organize it:

#### Suggested Module Breakdown:

1. `data.rs` – handles **reading/parsing** raw player data.
  - e.g., CSV reading, converting into structs
2. `stats.rs` – contains **logic for aggregating stats**.
  - e.g., summing points, rebounds, etc. across games
3. `leaderboard.rs` – handles **ranking logic**.
  - e.g., sorting players by totals, finding ranks
4. `main.rs` – acts as the program's entry point.
  - handles **user input/output**, calling functions from the other modules

This separation makes it easier to test, navigate, and scale your project.

Would you like help scaffolding out those files or writing the `mod` and `use` declarations?

- I defined a struct `PlayerGame` with fields that match the CSV columns and used `#[serde(rename = "...")]` to handle column name mismatches.
- I used the `csv` crate with `ReaderBuilder::new().from_path(...)` to read the CSV file.
- Each row was deserialized into a `PlayerGame` using a loop: `for result in rdr.deserialize()`, which leverages `serde` under the hood for efficient parsing.

I have a CSV file with NBA player stats like name, team, points, assists, etc. How can I load each row into a struct in Rust? What's the best way to parse that kind of data?

To parse a CSV file into structs in Rust, a common approach is to use the `csv` crate together with `serde` for automatic deserialization.

#### General Steps:

1. Define a struct that represents a single row of the CSV.
2. Use `#[derive(Deserialize)]` from `Serde`.
3. Use a CSV reader to read and deserialize each row.

#### Example:

```
rust
use serde::Deserialize;
use csv::ReaderBuilder;

#[derive(Debug, Deserialize)]
struct Record {
    #[serde(rename = "Name")]
    name: String,
    #[serde(rename = "Value")]
    value: Option<f32>,
}

fn load_data(path: &str) -> Result<Vec<Record>, Box<dyn std::error::Error>> {
    let mut rdr = ReaderBuilder::new().from_path(path)?;
    let mut records = Vec::new();

    for result in rdr.deserialize() {
        let row: Record = result?;
        records.push(row);
    }

    Ok(records)
}
```

This setup will automatically map each CSV row to a struct. Just make sure the field names match the CSV headers, or use `#[serde(rename = "...")]` to align them.

Let me know if you want help adjusting this to your specific CSV format.

- In `aggregate_stats()`, I used `entry.total_points += game.points.unwrap_or(0.0)` for every stat field that could potentially be missing.
- This allowed me to treat `None` values as zeros without panicking or needing long match statements.
- It kept the code concise, safe, and readable, especially when aggregating numerous games per player.

- I used `///` above every public function to write documentation comments in idiomatic Rust.
- I followed the `# Arguments` and `# Returns` structure to make the comments standardized and informative.
- This approach helped clarify function purpose and usage for each module, improving readability and maintainability.

I'm aggregating player stats, but some fields like points or assists might be missing (i.e., `None`). What's the cleanest way to handle that when summing values?

Great question. When dealing with `Option<f32>` (or any optional numeric type), a clean and idiomatic way to sum the values is to use `.unwrap_or(0.0)` — this treats missing data as zero during aggregation.

#### Example:

```
rust
let total_points = game.points.unwrap_or(0.0);
let total_assists = game.assists.unwrap_or(0.0);
```

You can then safely add these to your totals:

```
rust
player.total_points += total_points;
player.total_assists += total_assists;
```

This avoids `panic!` from unwrapping a `None` and keeps your stats accurate by defaulting missing values to zero. Let me know if you're doing this in a loop and want to streamline it!

What's the proper way to write doc comments in Rust? Should I use `///` or `/** */`? And is there a standard format for listing arguments and return values?

In Rust, the proper way to write doc comments is to use triple slashes: `///`.

#### ✓ Use This:

```
rust
/// Brief summary of what the function does.
///
/// # Arguments
/// * `x` - Description of parameter x.
/// * `y` - Description of parameter y.
///
/// # Returns
/// A description of what the function returns.
fn example(x: i32, y: i32) -> i32 {
    x + y
}
```

#### ✗ Avoid This (Java-style):

```
rust
/** This is not the idiomatic Rust way to document functions */
```

#### Notes:

- Rust uses **Markdown** inside doc comments, so you can include formatting like **`**bold**`**, code blocks, lists, etc.
- `///` is for documenting items (functions, structs, etc.). Use `//!` for module-level or crate-level documentation.
- The `# Arguments` and `# Returns` format isn't required, but it's a widely followed convention — especially for public APIs or educational code.

Want help writing doc comments for your own functions?